

AOV1test

Šedá je taky BaRvA

April 2026

1 Úloha obchodního cestujícího (Traveling Salesman Problem - TSP)

Úloha obchodního cestujícího patří mezi klasické diskrétní optimalizační problémy v teorii grafů a operačním výzkumu. Jejím cílem je najít nejkratší uzavřenou cestu (Hamiltonův cyklus), která navštíví všechny zadané uzly právě jednou a vrátí se do výchozího místa.

1.1 Základní matematický model TSP

Základní model předpokládá, že máme n uzlů (měst) a známe vzdálenosti c_{ij} mezi každou dvojicí uzlů i a j .

Rozhodovací proměnné:

- $x_{ij} = 1$, pokud cestující jede přímo z uzlu i do uzlu j .
- $x_{ij} = 0$ v opačném případě.
- u_i jsou pomocné proměnné pro eliminaci podcyklů (určují pořadí návštěvy uzlu i).

Formulace modelu:

$$\min z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \quad (1)$$

za podmínek:

$$\sum_{j=1, j \neq i}^n x_{ij} = 1, \quad i = 1, \dots, n \quad (2)$$

$$\sum_{i=1, i \neq j}^n x_{ij} = 1, \quad j = 1, \dots, n \quad (3)$$

$$\begin{aligned} u_i - u_j + n x_{ij} &\leq n - 1, \quad i, j = 2, \dots, n, i \neq j \\ x_{ij} &\in \{0, 1\}, \quad i, j = 1, \dots, n \end{aligned} \quad (4)$$

Interpretace prvků modelu:

- **Účelová funkce (1):** Minimalizuje celkovou ujetou vzdálenost (součet ohodnocení hran v Hamiltonově cyklu).
- **Podmínky (2) a (3):** Zajišťují, že z každého uzlu se právě jednou vyjede a do každého uzlu se právě jednou vjede.
- **Podmínka (4) (Miller-Tucker-Zemlin):** Zabraňuje vzniku tzv. podcyklů (uzavřených cest, které neprocházejí všemi uzly). Pokud by existoval podcyklus, nebylo by možné těmto uzlům přiřadit vzestupné pořadí u_i .

1.2 Model TSP s časovými okny (TSP-TW)

V praxi je často nutné navštívit zákazníka v konkrétním čase. K modelu se přidává časové okno $\langle a_i, b_i \rangle$ pro každý uzel i .

Doplňující proměnné a parametry:

- t_i : čas, ve kterém je navštíven uzel i .
- d_{ij} : doba přejezdu mezi uzly i a j .
- M : dostatečně velké kladné číslo (velká konstanta).

Dodatečná omezení:

$$a_i \leq t_i \leq b_i, \quad i = 1, \dots, n \quad (5)$$

$$t_i + d_{ij} - M(1 - x_{ij}) \leq t_j, \quad i, j = 1, \dots, n \quad (6)$$

Interpretace: Omezení (5) vynucuje, aby návštěva uzlu proběhla v intervalu od a_i do b_i . Podmínka (6) zajišťuje časovou následnost: pokud jedeme z i do j ($x_{ij} = 1$), musí být čas v j alespoň o dobu přejezdu vyšší než v i .

1.3 Formulace v jazyce LINGO a interpretace řešení

V programu LINGO se TSP zapisuje pomocí množin (SETS). Následující kód odpovídá zadání se 4 uzly:

MODEL:

SETS:

uzel /1..4/: u;

hrana(uzel, uzel): c, x;

ENDSETS

DATA:

c = 0 11 27 12

11 0 33 22

27 33 0 29

12 22 29 0;

n = @SIZE(uzel);

ENDDATA

min = @SUM(hrana: c * x);

@FOR(uzel(i): @SUM(uzel(j): x(i,j)) = 1);

@FOR(uzel(j): @SUM(uzel(i): x(i,j)) = 1);

@FOR(hrana(i,j) | j #GT# 1 #AND# i #NE# j:

u(i) + 1 - n * (1 - x(i,j)) <= u(j));

@FOR(hrana: @BIN(x));

END

Komentář k LINGO modelu:

- uzel /1..4/: u; definuje 4 uzly a pomocnou proměnnou u pro každý z nich.
- @BIN(x) určuje, že proměnná x je binární (0 nebo 1).
- #GT# 1 v podmínce pro u vynechává výchozí uzel (depot), což je nutné pro správnou funkci MTZ podmínek.

Interpretace řešení: Výsledkem je hodnota **Objective value**, která udává minimální délku trasy. Proměnné $x(i, j)$ s hodnotou 1 pak definují optimální trasu (např. $1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 1$).

1.4 Heuristiky pro řešení TSP

Vzhledem k tomu, že TSP je NP-těžký problém, pro velké sítě se používají přibližné (heuristické) metody, které rychle najdou přípustné, i když ne nutně optimální řešení.

1. Metoda nejbližšího souseda (Nearest Neighbor):

1. Začni v libovolném uzlu (např. uzel 1).
2. Ze všech dosud nenavštívených uzlů vyber ten, který má nejkratší vzdálenost od aktuálního uzlu.
3. Přesuň se do tohoto uzlu a označ jej za navštívený.
4. Opakuj kroky 2 a 3, dokud nenavštívíš všechny uzly.

5. Z posledního uzlu se vrať do výchozího bodu.

2. Vkládací metoda (Insertion Method): Principem je postupné rozšiřování malého podcyklu.

1. Vytvoř počáteční cyklus (např. mezi dvěma nejbližšími uzly).
2. Vyber nenavštívený uzel k (např. ten, který je nejbližší k některé z hran cyklu).
3. Vlož uzel k mezi uzly i a j aktuálního cyklu tak, aby nárůst délky trasy ($c_{ik} + c_{kj} - c_{ij}$) byl minimální.
4. Opakuj, dokud nejsou vloženy všechny uzly.

1.5 Rozbor konkrétního příkladu úlohy obchodního cestujícího (TSP)

V této podsekcí rozebereme konkrétní úlohu, ve které spotřebitel plánuje navštívit tři různé obchody a následně se vrátit domů. Cílem je minimalizovat celkovou ujetou vzdálenost.

1. Definice prvků a vstupních dat

Uvažujme graf se 4 uzly ($n = 4$), kde:

- **Uzel 1 (VB):** Výchozí místo zákazníka (depot).
- **Uzel 2 (OB1):** První obchod.
- **Uzel 3 (OB2):** Druhý obchod.
- **Uzel 4 (OB3):** Třetí obchod.

Vzdálenosti mezi těmito místy jsou dány následující maticí vzdáleností C (v km):

$$C = \begin{bmatrix} 0 & 11 & 27 & 12 \\ 11 & 0 & 33 & 22 \\ 27 & 33 & 0 & 29 \\ 12 & 22 & 29 & 0 \end{bmatrix} \quad (7)$$

2. Sestavení matematického modelu

K vyřešení této úlohy definujeme rozhodovací proměnné a omezující podmínky tak, aby výsledkem byl jediný uzavřený okruh procházející všemi uzly.

Rozhodovací proměnné:

- $x_{ij} \in \{0, 1\}$: Binární proměnná. Má hodnotu 1, pokud cestující přejíždí přímo z místa i do místa j . Pokud je $x_{12} = 1$, znamená to, že první cesta vede z domova do obchodu OB1.
- $u_i \in \mathbb{R}$: Pomocné proměnné pro uzly $i = 2, 3, 4$, které slouží k eliminaci tzv. podcyklů (zajišťují, že trasa nebude rozdělena na několik malých uzavřených smyček).

Účelová funkce: Minimalizujeme součet vzdáleností všech vybraných úseků:

$$\min z = \sum_{i=1}^4 \sum_{j=1}^4 c_{ij} x_{ij} \quad (8)$$

Vlastní omezení:

- **Výstupní podmínky:** Z každého místa musíme vyjet právě jednou:

$$\sum_{j=1, j \neq i}^4 x_{ij} = 1, \quad i = 1, \dots, 4 \quad (9)$$

- **Vstupní podmínky:** Do každého místa musíme přijet právě jednou:

$$\sum_{i=1, i \neq j}^4 x_{ij} = 1, \quad j = 1, \dots, 4 \quad (10)$$

- **Podmínky pro eliminaci podcyklů (MTZ):** Pro všechny dvojice uzlů mimo výchozí místo ($i, j \in \{2, 3, 4\}, i \neq j$):

$$u_i - u_j + 4x_{ij} \leq 3 \quad (11)$$

3. Ukázka a interpretace řešení

Předpokládejme, že po výpočtu získáme následující hodnoty proměnných $x_{ij} = 1$:

- $x_{12} = 1$ (Cesta z VB do OB1)
- $x_{23} = 1$ (Cesta z OB1 do OB2)
- $x_{34} = 1$ (Cesta z OB2 do OB3)
- $x_{41} = 1$ (Cesta z OB3 zpět do VB)

Výpočet délky trasy: Dosadíme odpovídající vzdálenosti z matice C :

$$z = c_{12} + c_{23} + c_{34} + c_{41} \quad (12)$$

$$z = 11 + 33 + 29 + 12 = \mathbf{85 \text{ km}} \quad (13)$$

Interpretace řešení: Tento výsledek definuje konkrétní okruh $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$. Zákazník tedy nejdříve navštíví první obchod, odtud pokračuje do druhého, následně do třetího a z něj se vrací domů. Celkem ujede 85 km. Matematický model v tomto případě zajistil, že nebyl vybrán např. pouze krátký okruh mezi domovem a prvním obchodem ($1 \rightarrow 2 \rightarrow 1$), protože podmínky u_i by takové řešení nepovolily.

3. Metoda výhodnostních čísel (Savings Method): Tato metoda se častěji používá pro rozvozní úlohy, ale lze ji aplikovat i na TSP. Spočívá v tom, že uvažujeme návštěvy uzlů "hvězdovitě" z depa ($1 \rightarrow i \rightarrow 1$) a následně spojujeme trasy tam, kde je nejvyšší úspora (výhodnostní číslo $s_{ij} = c_{1i} + c_{1j} - c_{ij}$). (Poznámka: Informace o postupu této metody není v přímých zdrojích detailně popsána krok za krokem, vychází z obecných principů heuristik).

2 Rozvozní úloha (Vehicle Routing Problem - VRP)

Rozvozní úloha je zobecněním úlohy obchodního cestujícího. Zatímco u TSP hledáme jedinou trasu pro jedno vozidlo bez ohledu na jeho náklad, VRP řeší situaci, kdy má vozidlo omezenou kapacitu a musí obsloužit zákazníky s různě velkými požadavky. Cílem je minimalizovat celkovou ujetou vzdálenost všech tras.

2.1 Základní matematický model VRP

Základní model uvažuje jedno výchozí místo (depot), sadu zákazníků s požadavky q_i a vozidlo s pevnou kapacitou V .

Rozhodovací proměnné:

- $x_{ij} = 1$, pokud vozidlo přejíždí přímo z uzlu i do uzlu j , jinak 0.
- u_i : pomocná proměnná vyjadřující aktuální vytížení vozidla po obsluze uzlu i .

Matematická formulace:

$$\min z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \quad (14)$$

za podmínek:

$$\sum_{j=1, j \neq i}^n x_{ij} = 1, \quad i = 2, \dots, n \quad (15)$$

$$\sum_{i=1, i \neq j}^n x_{ij} = 1, \quad j = 2, \dots, n \quad (16)$$

$$u_i + q_j - V(1 - x_{ij}) \leq u_j, \quad i \in \{1, \dots, n\}, j \in \{2, \dots, n\}, i \neq j \quad (17)$$

$$q_i \leq u_i \leq V, \quad i = 2, \dots, n \quad (18)$$

$$u_1 = 0, \quad x_{ij} \in \{0, 1\} \quad (19)$$

Interpretace prvků modelu:

- **Účelová funkce:** Minimalizuje součet délek všech tras (hran), které vozidla projedou.
- **Vstupní a výstupní podmínky:** Zajišťují, že každý zákazník (uzly 2 až n) je obsloužen právě jednou. Pro depot (uzel 1) platí, že z něj může vyjet více tras a více se jich do něj vrátit.
- **Kapacitní omezení (MTZ varianta):** Proměnná u_i sleduje náklad ve vozidle. Podmínka zajišťuje, že pokud jedeme z i do j , musí se náklad zvýšit o požadavek q_j . Zároveň tato podmínka brání vzniku podcyklů, které by neprocházely depotem.

2.2 Model s časovými okny (VRP-TW)

Tento model rozšiřuje základní VRP o časové omezení, kdy zákazník i může být obsloužen pouze v intervalu $\langle a_i, b_i \rangle$.

Nové prvky modelu:

- t_i : čas návštěvy uzlu i .
- d_{ij} : doba přejezdu mezi uzly.
- S_i : doba obsluhy (nakládky/vykládky) v uzlu i .
- M : velké kladné číslo (konstanta).

Dodatečná omezení:

$$a_i \leq t_i \leq b_i, \quad i = 2, \dots, n \quad (20)$$

$$t_i + S_i + d_{ij} - M(1 - x_{ij}) \leq t_j, \quad i, j = 1, \dots, n \quad (21)$$

$$t_1 = 0 \quad (22)$$

Rozdíl oproti základnímu modelu: Zatímco základní VRP hlídá pouze, aby se do auta vešlo zboží (hmotnost/objem), VRP-TW navíc koordinuje logistiku v čase. Pokud vozidlo dorazí dříve než v čase a_i , musí čekat. Pokud by dorazilo po b_i , řešení je nepřipustné.

2.3 Formulace v jazyce LINGO

Zápis v LINGO využívá sady (SETS) pro definici uzlů a hran. Následující ukázka odpovídá základnímu modelu:

MODEL:

SETS:

uzel /1..4/: u, q;

hrana(uzel, uzel): c, x;

ENDSETS

DATA:

c = 0 11 27 12

11 0 33 22

27 33 0 29

12 22 29 0;

q = 0 2 8 9;

V = 11;

ENDDATA

min = @SUM(hrana: c * x);

@FOR(uzel(i) | i #GT# 1: @SUM(uzel(j): x(i,j)) = 1);

@FOR(uzel(j) | j #GT# 1: @SUM(uzel(i): x(i,j)) = 1);

@FOR(hrana(i,j) | j #GT# 1: u(i) + q(j) - V * (1 - x(i,j)) <= u(j));

@FOR(uzel(i) | i #GT# 1: q(i) <= u(i); u(i) <= V);

u(1) = 0;

@FOR(hrana: @BIN(x));

END

Komentář k modelu:

- $q(i) \leq u(i)$; $u(i) \leq V$ zajišťuje, že v každém bodě trasy náklad nepřekročí nosnost vozidla 11.
- $i \#GT\# 1$ vynechává depot z podmínek pro jednoznačný vjezd/výjezd, protože z depa vyjíždí všechna použitá vozidla.
- $@BIN(x)$ definuje binární proměnné (jede/nejede).

2.4 Konkrétní číselný příklad

Mějme depot (1) a tři obchody (2, 3, 4).

- **Požadavky:** $q_2 = 2$ t, $q_3 = 8$ t, $q_4 = 9$ t.
- **Kapacita vozidla:** $V = 11$ t.
- **Vzdálenosti:** $c_{12} = 11$, $c_{13} = 27$, $c_{14} = 12$, $c_{23} = 33$, $c_{24} = 22$, $c_{34} = 29$ (matice je symetrická).

Analýza: Celkový požadavek je $2 + 8 + 9 = 19$ tun. Protože kapacita vozidla je pouze 11 tun, musí být vyslána minimálně dvě vozidla (nebo jedno vozidlo musí jet dvě trasy).

Formulace a výsledek: Model v LINGO propočítá kombinace vjezdů a výjezdů. Vzhledem k vysokým požadavkům obchodů 3 a 4 (dohromady $17 > 11$) nemohou být na stejné trase. Obchod 2 (2 t) se může přidat buď k obchodu 3 ($2 + 8 = 10 \leq 11$), nebo k obchodu 4 ($2 + 9 = 11 \leq 11$).

Interpretace výsledného řešení: Při řešení vyjdou proměnné $x_{ij} = 1$ pro tyto hrany:

- Trasa 1: $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$ (Vzdálenost: $11 + 33 + 27 = 71$)
- Trasa 2: $1 \rightarrow 4 \rightarrow 1$ (Vzdálenost: $12 + 12 = 24$)
- **Celková vzdálenost:** 95.

Slovně: První vozidlo odveze 10 tun zboží pro obchody 2 a 3. Druhé vozidlo odveze 9 tun pro obchod 4. Obě vozidla startují i končí v depu. Tato varianta je optimální, protože jakékoli jiné spojení (např. $2 + 4$) by vedlo k delší celkové trase.

3 Úloha čínského listonoše (Chinese Postman Problem - CPP)

Úloha čínského listonoše řeší problém nalezení nejkratší uzavřené trasy v grafu, která obsahuje každou hranu (ulici) alespoň jednou. Na rozdíl od úlohy obchodního cestujícího, kde je cílem navštívit všechny **uzly**, zde se zaměřujeme na obsluhu všech **hran**.

3.1 Základní principy a Eulerovy cykly

Intuitivní vysvětlení: Představme si listonoše, který musí projít všechny ulice ve svém rajónu a vrátit se na poštu. Ideálním stavem je, pokud může každou ulici projít právě jednou. To je možné pouze tehdy, pokud graf obsahuje tzv. **Eulerův cyklus**.

- **Neorientovaný graf:** Eulerův cyklus existuje, pokud jsou všechny uzly sudého stupně (do každé křižovatky vchází sudý počet ulic).
- **Orientovaný graf:** Eulerův cyklus existuje, pokud pro každý uzel platí, že počet vstupujících hran se rovná počtu vystupujících hran.

Pokud tyto podmínky nejsou splněny, musí listonoš některé ulice projít vícekrát (duplikovat hrany). Matematické modely CPP hledají, které hrany zduplikovat, aby celkový nárůst trasy byl minimální.

3.2 Matematické modely pro neorientovaný graf

Pro neorientované grafy se využívají dva základní přístupy k formulaci.

3.2.1 Model I: Výběr hran ke zdvojení

Tento model se zaměřuje na identifikaci konkrétních hran, které musíme přidat do grafu, aby se všechny uzly staly sudými.

Matematická formulace:

$$\min z = \sum_{i=1}^{n-1} \sum_{j=i+1}^n c_{ij} x_{ij} \quad (23)$$

za podmínek:

$$\sum_{j < i} x_{ji} + \sum_{j > i} x_{ij} = 2y_i + 1, \quad \forall i \in T \quad (24)$$

$$\sum_{j < i} x_{ji} + \sum_{j > i} x_{ij} = 2y_i, \quad \forall i \in U \setminus T \quad (25)$$

$$x_{ij} \in \{0, 1\}, \quad y_i \geq 0, y_i \in \mathbb{Z} \quad (26)$$

Interpretace:

- **Sety:** T je množina uzlů s lichým stupněm, $U \setminus T$ jsou uzly se sudým stupněm.
- **Proměnná x_{ij} :** Binární proměnná udávající, zda do grafu přidáme (zduplikujeme) hranu mezi uzly i a j .
- **Podmínky:** Pro liché uzly musí být počet přidanych hran lichý (např. $2y_i + 1$), čímž se jejich celkový stupeň změní na sudý. Pro sudé uzly musí být počet přidanych hran sudý, aby sudými zůstaly.

3.2.2 Model II: Určení počtu průchodů

Tento model přímo určuje, kolikrát bude každá hrana v grafu využita.

Matematická formulace:

$$\min z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \quad (27)$$

za podmínek:

$$\sum_{j=1}^n x_{ij} - \sum_{j=1}^n x_{ji} = 0, \quad \forall i \in \{1, \dots, n\} \quad (28)$$

$$x_{ij} + x_{ji} \geq 1, \quad \forall (i, j) \in H \quad (29)$$

$$x_{ij} \geq 0, \quad x_{ij} \in \mathbb{Z} \quad (30)$$

Interpretace:

- **Proměnná x_{ij} :** Udává, kolikrát projdeme hranu z i do j .
- **Podmínka $x_{ij} + x_{ji} \geq 1$:** Zajišťuje, že každá hrana původního grafu H bude využita alespoň v jednom směru.
- **Rovnováha toku:** První podmínka simuluje Eulerovu vlastnost – kolikrát do uzlu vstoupíme, tolikrát musíme vystoupit.

3.3 Matematický model pro orientovaný graf

V orientovaném grafu řešíme CPP jako speciální případ dopravního problému, kde vyrovnáváme deficity vstupujících a vystupujících hran.

Matematická formulace:

$$\min z = \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij} \quad (31)$$

za podmínek:

$$\sum_{j \in J} x_{ij} = a_i, \quad \forall i \in I \quad (32)$$

$$\sum_{i \in I} x_{ij} = b_j, \quad \forall j \in J \quad (33)$$

$$x_{ij} \geq 0 \quad (34)$$

Interpretace prvků:

- **Množina I :** Uzly, kde počet vstupujících hran převyšuje vystupující ($d_{in} > d_{out}$). Hodnota a_i je tento rozdíl.
- **Množina J :** Uzly, kde počet vystupujících hran převyšuje vstupující ($d_{out} > d_{in}$). Hodnota b_j je tento rozdíl.
- **Cíl:** Propojit uzly "přebytku" s uzly "nedostatku" pomocí cest s minimální cenou c_{ij} , čímž se graf stane eulerovským.

3.4 Implementace v jazyce LINGO a interpretace řešení

Následující příklad ukazuje implementaci **Modelu II** pro neorientovaný graf (5 uzlů):

MODEL:

SETS:

uzel /1..5/;

hrana(uzel,uzel) /1 2, 1 3, 1 5, 2 1, 2 3, 3 1, 3 2,

```

3 4, 4 3, 4 5, 5 1, 5 4/: x, c;
ENDSETS
DATA:
c = 3 5 1 3 4 5 4 8 8 10 1 10;
ENDDATA
min = @SUM(hrana: c * x);
@FOR(hrana(i,j): x(i,j) + x(j,i) >= 1);
@FOR(uzel(i): @SUM(hrana(i,j): x(j,i)) - @SUM(hrana(i,j): x(i,j)) = 0);
@FOR(hrana: @GIN(x));
END

```

Komentář k modelu:

- **hrana(uzel,uzel) /.../:** Definuje seznam existujících ulic v obou směrech (pro neorientovaný graf zadáváme obě orientace).
- **x(i,j) + x(j,i) >= 1:** Příkaz "projít každou ulici alespoň jednou" (buď tam, nebo zpět).
- **@GIN(x):** Zajišťuje, že počet průchodů bude celé číslo.

Interpretace výsledku: Po spuštění LINGO získáme hodnoty x_{ij} . Pokud např. $x_{12} = 1$ a $x_{21} = 1$, znamená to, že hranu mezi uzly 1 a 2 projdeme jednou tam a jednou zpět (zdvojení). Pokud $x_{ij} = 1$ a $x_{ji} = 0$, projdeme hranu pouze jednou. Celková hodnota **Objective value** udává minimální délku obchůzky.

3.5 Příklad a duplicity hran

Mějme jednoduchý graf čtverce s jednou úhlopříčkou.

- **Analýza stupňů:** Uzly na koncích úhlopříčky budou mít stupeň 3 (lichý). Zbylé dva uzly stupeň 2 (sudý).
- **Řešení duplicity:** Model identifikuje, že nejlevnější cestou k "uzdravení" lichých uzlů je zduplikovat úhlopříčku (pokud je nejkratší) nebo zduplikovat dvě sousední strany čtverce.
- **Výsledek:** V maticové reprezentaci řešení x_{ij} uvidíme u zduplikované hrany hodnotu 2 (resp. součet $x_{ij} + x_{ji} = 2$), což znamená, že tudy listonoš půjde dvakrát.

3.6 Rozbor konkrétního příkladu úlohy čínského listonoše (CPP)

V této části podrobně rozebereme aplikaci úlohy čínského listonoše na konkrétní silniční síť. Cílem listonoše je projít všechny ulice (hrany) alespoň jednou a vrátit se do výchozího bodu s minimální celkovou ujetou vzdáleností.

1. Definice sítě a vstupních dat

Uvažujme město s 5 křižovatkami (uzly) a systémem ulic (hran). Pro výpočet využijeme data z matice vzdáleností, kde hodnota ∞ značí, že mezi uzly neexistuje přímé spojení.

Matice vzdáleností C (v km):

$$C = \begin{bmatrix} 0 & 3 & 5 & \infty & 1 \\ 3 & 0 & 4 & \infty & \infty \\ 5 & 4 & 0 & 8 & \infty \\ \infty & \infty & 8 & 0 & 10 \\ 1 & \infty & \infty & 10 & 0 \end{bmatrix} \quad (35)$$

Tato matice reprezentuje následující hrany v neorientovaném grafu:

- $h_{1,2}$ s délkou 3

- $h_{1,3}$ s délkou 5
- $h_{1,5}$ s délkou 1
- $h_{2,3}$ s délkou 4
- $h_{3,4}$ s délkou 8
- $h_{4,5}$ s délkou 10

2. Sestavení matematického modelu (Model II)

Pro řešení využijeme model, který určuje počet průchodů jednotlivými hranami v obou směrech.

Rozhodovací proměnné:

- $x_{ij} \in \mathbb{Z}^+$: Celočíslná proměnná udávající, kolikrát listonoš projde hranu z uzlu i do uzlu j .

Účelová funkce: Minimalizujeme celkovou délku trasy:

$$\min z = \sum_{i=1}^5 \sum_{j=1}^5 c_{ij} x_{ij} \quad (36)$$

Omezující podmínky:

- **Podmínka pokrytí všech hran:** Každá existující ulice v grafu H musí být projeta alespoň jednou (v libovolném směru):

$$x_{ij} + x_{ji} \geq 1, \quad \forall (i, j) \in H \quad (37)$$

- **Podmínka rovnováhy (Eulerova vlastnost):** V každém uzlu se musí počet vstupů rovnat počtu výstupů, aby trasa byla uzavřená:

$$\sum_{j=1}^5 x_{ji} - \sum_{j=1}^5 x_{ij} = 0, \quad i = 1, \dots, 5 \quad (38)$$

3. Interpretace a analýza řešení

Analýza stupňů uzlů: Před výpočtem zjistíme stupně uzlů v původním grafu:

- Uzel 1: Sousedí s 2, 3, 5 $\rightarrow$ Stupeň 3 (**lichý**)
- Uzel 2: Sousedí s 1, 3 $\rightarrow$ Stupeň 2 (sudý)
- Uzel 3: Sousedí s 1, 2, 4 $\rightarrow$ Stupeň 3 (**lichý**)
- Uzel 4: Sousedí s 3, 5 $\rightarrow$ Stupeň 2 (sudý)
- Uzel 5: Sousedí s 1, 4 $\rightarrow$ Stupeň 2 (sudý)

Protože uzly 1 a 3 mají lichý stupeň, neexistuje v původním grafu Eulerův cyklus. Listonoš bude muset některou hranu (nebo cestu) projít dvakrát, aby "uzdravil" liché stupně uzlů.

Možné řešení: Model identifikuje nejlevnější spojení mezi lichými uzly 1 a 3. Nejkratší cesta je přímá hrana $h_{1,3}$ s délkou 5.

- Pro hranu (1,3) bude platit $x_{13} + x_{31} = 2$ (průjezd tam i zpět).
- Pro ostatní hrany bude platit $x_{ij} + x_{ji} = 1$ (průjezd pouze jednou).

Interpretace výsledku: Celková délka trasy bude součet délek všech hran (31 km) plus délka zdvojené cesty (5 km), tedy **36 km**. Listonoš projede trasu například v pořadí: **1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 4 $\rightarrow$ 5 $\rightarrow$ 1 $\rightarrow$ 3 $\rightarrow$ 1**. Všimněte si, že díky zdvojení hrany (1,3) se listonoš mohl vrátit do uzlu 1 a zároveň obsloužit úsek mezi 1 a 3, který mu v původním Eulerově průchodu chyběl.

4 Alokační problém (Allocation Problem)

Alokační problém je speciálním typem úloh lineárního programování, který řeší optimální rozdělení omezených zdrojů (kapacit) mezi různé požadavky tak, aby byly minimalizovány celkové náklady nebo maximalizován celkový užitek. V praxi se nejčastěji setkáváme s přiřazením dodavatelů k odběratelům, kde každý odběratel musí být obslužen právě jedním dodavatelem (tzv. nedělitelná dodávka).

4.1 Matematický model alokačního problému

Model vychází z předpokladu, že máme m dodavatelů s omezenými kapacitami a n odběratelů s pevně danými požadavky.

Rozhodovací proměnné:

- $x_{ij} \in \{0, 1\}$: Binární proměnná, která nabývá hodnoty 1, pokud dodavatel i zásobuje odběratele j , v opačném případě je 0.

Parametry modelu:

- c_{ij} : Náklady na přepravu jednotky produkce od dodavatele i k odběrateli j .
- a_i : Kapacita i -tého dodavatele.
- b_j : Velikost požadavku j -tého odběratele.

Formulace modelu:

$$\min z = \sum_{i=1}^m \sum_{j=1}^n c_{ij} b_j x_{ij} \quad (39)$$

za podmínek:

$$\sum_{i=1}^m x_{ij} = 1, \quad j = 1, \dots, n \quad (40)$$

$$\sum_{j=1}^n b_j x_{ij} \leq a_i, \quad i = 1, \dots, m \quad (41)$$

$$x_{ij} \in \{0, 1\}, \quad i = 1, \dots, m, \quad j = 1, \dots, n \quad (42)$$

4.2 Interpretace matematického modelu

- **Účelová funkce:** Vyjadřuje snahu o minimalizaci celkových nákladů. Součin $c_{ij} b_j x_{ij}$ představuje náklady na obsluhu celého požadavku odběratele j dodavatelem i .
- **Podmínka obsluhy odběratele:** Suma přes index i (dodavatelé) rovna 1 zajišťuje, že každý odběratel j dostane svou zásilku od **právě jednoho** dodavatele.
- **Kapacitní omezení:** Suma požadavků všech odběratelů přiřazených danému dodavateli i nesmí překročit jeho celkovou kapacitu a_i .
- **Nedělitelnost (Binární podmínka):** Proměnná x_{ij} musí být celočíselná (0 nebo 1), což v praxi znamená, že dodávku nelze dělit mezi více dodavatelů.

4.3 Formulace modelu v jazyce LINGO

V jazyce LINGO využíváme sekce SETS pro definici struktur a DATA pro vložení konkrétních hodnot.

MODEL:

SETS:

```
dodavatel /1..3/: capacity;  
odberatel /1..10/: požadavky;
```

```

    trasa(dodavatel, odberatel): naklady, x;
ENDSETS

DATA:
    kapacita = 5000, 7000, 6000;
    pozadavky = 2000, 1000, 800, 700, 900, 2200, 1800, 1700, 1100, 1200;
    naklady = 72 46 85 26 63 41 35 96 86 73
              62 75 27 23 46 55 53 61 77 96
              51 82 13 54 35 80 60 67 42 59;
ENDDATA

min = @SUM(trasa(i,j): x(i,j) * naklady(i,j) * pozadavky(j));

@FOR(odberatel(j):
    @SUM(dodavatel(i): x(i,j)) = 1
);

@FOR(dodavatel(i):
    @SUM(odberatel(j): x(i,j) * pozadavky(j)) <= kapacita(i)
);

@FOR(trasa: @BIN(x));
END

```

Komentář k LINGO modelu:

- **SETS:** Definuje dodavatele (index i) a odběratele (index j). Relace **trasa** propojuje oba a nese atributy nákladů a rozhodovací proměnné x .
- **min:** Definice účelové funkce jako sumy přes všechny trasy.
- **@FOR(odberatel(j)):** Zajišťuje, že pro každého odběratele je součet jeho binárních proměnných roven 1 (vždy jeden dodavatel).
- **@BIN(x):** Klíčový příkaz, který mění úlohu z lineární na celočíselnou (binární).

4.4 Konkrétní číselný příklad

Uvažujme malou instanci se 2 dodavateli a 3 odběrateli.

Vstupní data:

- Kapacity dodavatelů: $a_1 = 100$, $a_2 = 150$.
- Požadavky odběratelů: $b_1 = 80$, $b_2 = 70$, $b_3 = 60$.
- Matice nákladů c_{ij} (na jednotku):

$$C = \begin{bmatrix} 10 & 20 & 5 \\ 15 & 10 & 20 \end{bmatrix} \quad (43)$$

Matematická formulace účelové funkce pro tento příklad:

$$\begin{aligned} \min z = & (10 \cdot 80 \cdot x_{11}) + (20 \cdot 70 \cdot x_{12}) + (5 \cdot 60 \cdot x_{13}) + \dots \\ & \dots + (15 \cdot 80 \cdot x_{21}) + (10 \cdot 70 \cdot x_{22}) + (20 \cdot 60 \cdot x_{23}) \end{aligned} \quad (44)$$

Ukázka řešení a interpretace: Při výpočtu by model hledal kombinaci x_{ij} , která nepřekročí kapacity (100 a 150).

- Pokud $x_{13} = 1$ a $x_{11} = 1$, dodavatel 1 vyčerpá kapacitu $60 + 80 = 140$, což je **přes limit** (100).

- Přípustné řešení může být: $x_{13} = 1$ (dodavatel 1 obslouží odběratele 3, náklad 300) a $x_{21} = 1, x_{22} = 1$ (dodavatel 2 obslouží odběratele 1 a 2, náklad $1200 + 700 = 1900$).
- **Interpretace:** Odběratel 3 bude zásobován dodavatelem 1, protože má k němu nejnížší náklady. Zbylí odběratelé spadnou pod dodavatele 2, aby byla dodržena kapacitní omezení. Celkové náklady budou 2200.

5 Přiřazovací problém (Assignment Problem)

Přiřazovací problém je speciálním případem dopravního problému, kde hledáme optimální (nejčastěji nejlevnější) vzájemné přiřazení dvou množin o stejném počtu prvků n . Typickým příkladem v praxi je přiřazení n pracovníků na n různých pracovních činnostech, přiřazení strojů k operacím nebo firem k projektům tak, aby každý pracovník vykonával právě jednu činnost a každá činnost byla obsazena právě jedním pracovníkem.

5.1 Matematický model přiřazovacího problému

Model předpokládá čtvercovou matici nákladů (nebo zisků) typu $n \times n$. Cílem je vybrat v každém řádku a každém sloupci právě jeden prvek tak, aby jejich součet byl minimální.

Rozhodovací proměnné:

- $x_{ij} \in \{0, 1\}$: Binární proměnná, která nabývá hodnoty 1, pokud je i -tý subjekt přiřazen k j -té činnosti. V opačném případě je rovna 0.

Parametry modelu:

- c_{ij} : Náklady spojené s přiřazením i -tého subjektu k j -té činnosti.
- n : Celkový počet prvků v každé množině (počet subjektů i činností).

Formulace modelu:

$$\min z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \quad (45)$$

za podmínek:

$$\sum_{j=1}^n x_{ij} = 1, \quad i = 1, \dots, n \quad (46)$$

$$\sum_{i=1}^n x_{ij} = 1, \quad j = 1, \dots, n \quad (47)$$

$$x_{ij} \in \{0, 1\}, \quad i, j = 1, \dots, n \quad (48)$$

Interpretace matematického modelu:

- **Účelová funkce:** Minimalizuje celkové náklady vzniklé součtem nákladů vybraných přiřazení (c_{ij} tam, kde $x_{ij} = 1$).
- **První sada omezení (řádková):** Zajišťuje, že každý subjekt i je přiřazen k právě jedné činnosti.
- **Druhá sada omezení (sloupcová):** Zajišťuje, že každá činnost j je vykonávána právě jedním subjektem.
- **Podmínka binární volby:** Vylučuje dělení úkolů mezi více subjektů; subjekt buď úkol dělá celý, nebo vůbec.

5.2 Formulace modelu v jazyce LINGO

Zápis v LINGO využívá množiny pro subjekty (firmy) a objekty (projekty). Níže je uveden vzorový model pro $n = 5$.

MODEL:

SETS:

```
firmy /1..5/;  
projekty /1..5/;
```



```

    prirazeni(firmy, projekty): naklady, x;
ENDSETS

DATA:
    naklady =
        10000 14000 22000 30000 25000
        12000 13000 19000 23000 20000
        18000 15000 18000 20000 27000
        14000 17000 18000 21000 19000
        11000 17000 19000 26000 22000;
ENDDATA

min = @SUM(prirazeni(i,j): naklady(i,j) * x(i,j));

@FOR(firmy(i):
    @SUM(projekty(j): x(i,j)) = 1
);

@FOR(projekty(j):
    @SUM(firmy(i): x(i,j)) = 1
);

@FOR(prirazeni(i,j): @BIN(x(i,j)));
END

```

Komentář k LINGO modelu:

- **SETS:** Definuje dvě primitivní množiny a jednu odvozenou množinu **prirazeni**, která obsahuje matici nákladů a rozhodovacích proměnných.
- **min:** Definuje účelovou funkci jako skalární součin matic **naklady** a **x**.
- **@FOR:** První cyklus hlídá přiřazení všech firem, druhý cyklus obsazení všech projektů.
- **@BIN:** Definice proměnných jako binárních (0/1).

5.3 Konkrétní číselný příklad

Mějme 5 firem (F1–F5) a 5 projektů (P1–P5). Každá firma může realizovat pouze jeden projekt.

Vstupní matice nákladů C :

$$C = \begin{bmatrix} 10000 & 14000 & 22000 & 30000 & 25000 \\ 12000 & 13000 & 19000 & 23000 & 20000 \\ 18000 & 15000 & 18000 & 20000 & 27000 \\ 14000 & 17000 & 18000 & 21000 & 19000 \\ 11000 & 17000 & 19000 & 26000 & 22000 \end{bmatrix} \quad (49)$$

Sestavení modelu pro tento příklad: Minimalizujeme $z = 10000x_{11} + 14000x_{12} + \dots + 22000x_{55}$. Omezení např. pro Firmu 1: $x_{11} + x_{12} + x_{13} + x_{14} + x_{15} = 1$. Omezení např. pro Projekt 1: $x_{11} + x_{21} + x_{31} + x_{41} + x_{51} = 1$.

Interpretace možného řešení: Předpokládejme, že optimální řešení vybere tyto dvojice:

- $x_{11} = 1$ (Firma 1 $\rightarrow$ Projekt 1, náklady 10 000)
- $x_{22} = 1$ (Firma 2 $\rightarrow$ Projekt 2, náklady 13 000)
- $x_{34} = 1$ (Firma 3 $\rightarrow$ Projekt 4, náklady 20 000)

- $x_{45} = 1$ (Firma 4 $\rightarrow$ Projekt 5, náklady 19 000)
- $x_{53} = 1$ (Firma 5 $\rightarrow$ Projekt 3, náklady 19 000)

Slovní interpretace: Při tomto přiřazení je každá firma vytížena právě jedním projektem a celkové náklady na realizaci všech pěti projektů dosahují minimální možné hodnoty (v tomto případě 81 000). Jakákoliv jiná záměna firem by vedla ke zvýšení celkové částky. Výsledek z LINGO by tyto dvojice zobrazil jako proměnné s hodnotou 1.000000.

6 Úloha o pokrytí (Set Covering Problem)

Úloha o pokrytí se v operačním výzkumu využívá k nalezení optimálního umístění center služeb tak, aby byly obslouženy všechny požadované oblasti (body). Typickým praktickým využitím je rozmístění stanic záchranné služby, hasičských sborů, vysílačů nebo skladů, kde je cílem minimalizovat náklady nebo maximalizovat dostupnost pro všechny uživatele.

6.1 Matematický model úlohy o pokrytí

Model pracuje se sadou potenciálních míst pro vybudování center a sadou oblastí, které musí být pokryty.

Rozhodovací proměnné:

- $y_i \in \{0, 1\}$: Binární proměnná. Má hodnotu 1, pokud je v místě i vybudováno centrum (stanice), jinak 0.
- $x_{ij} \in \{0, 1\}$: Binární proměnná. Má hodnotu 1, pokud je oblast j obsluhována centrem v místě i , jinak 0.

Účelové funkce (dvě verze): Podle cíle analýzy lze zvolit různé typy účelových funkcí:

1. **Minimalizace počtu center (nebo nákladů na výstavbu):** Používá se v situacích, kdy je prioritou úspora zdrojů při splnění podmínky dostupnosti.

$$\min z = \sum_{i=1}^n c_i y_i \quad (50)$$

Kde c_i představuje fixní náklady na zřízení centra i . Pokud jsou náklady stejné, minimalizujeme prostý počet stanic $\sum y_i$.

2. **Minimalizace celkové obslužné vzdálenosti (času):** Zaměřuje se na kvalitu služby (např. dojezdový čas záchrany) váženou četností požadavků f_j v daných oblastech.

$$\min z = \sum_{i=1}^n \sum_{j=1}^m c_{ij} f_j x_{ij} \quad (51)$$

Kde c_{ij} je dojezdový čas mezi místem i a j .

Omezující podmínky:

- **Podmínka plného pokrytí:** Každá oblast j musí být obsloužena právě jedním centrem:

$$\sum_{i=1}^n x_{ij} = 1, \quad j = 1, \dots, m \quad (52)$$

- **Logická vazba mezi centrem a obsluhou:** Oblast j může být obsluhována z místa i pouze tehdy, pokud je v místě i centrum skutečně vybudováno:

$$x_{ij} \leq y_i, \quad \forall i, j \quad (53)$$

(V praktických modelech LINGO se často používá sumární zápis $\sum_j x_{ij} \leq M y_i$, kde M je dostatečně velké číslo, např. počet všech oblastí.)

- **Počet center:** Často je dán pevný počet center K , která má magistrát k dispozici:

$$\sum_{i=1}^n y_i = K \quad (54)$$

6.2 Formulace v jazyce LINGO

Model v LINGO využívá sady pro oblasti a potenciální stanice. Následující zápis odpovídá minimalizaci vážených dojezdových časů pro 5 oblastí při stavbě 2 stanic.

MODEL:

SETS:

```
obvody /1..5/: f;           ! cetnosti pozadavku;
stanice /1..5/: y;          ! binarni promenna pro stavbu;
poloha(stanice, obvody): x, c; ! x=prirazeni, c=dojezdovy cas;
```

ENDSETS

DATA:

```
f = 30 50 42 36 24;
c = 4 12 14 17 11
    20 7 10 19 24
    21 13 5 8 11
    9 12 14 3 8
    17 25 13 10 6;
```

ENDDATA

! Ucelova funkce: minimalizace vazeneho dojezdoveho casu;

min = @SUM(poloha(i,j): c(i,j) * x(i,j) * f(j));

! Kazdy obvod musi byt obslouzen;

@FOR(obvody(j): @SUM(stanice(i): x(i,j)) = 1);

! Obvod muze obsluhovat jen existujici stanice (kapacita omezena na 4);

@FOR(stanice(i): @SUM(obvody(j): x(i,j)) <= 4 * y(i));

! Musime vybudovat prave 2 stanice;

@SUM(stanice(i): y(i)) = 2;

@FOR(stanice: @BIN(y));

@FOR(poloha: @BIN(x));

END

6.3 Rozbor konkrétního příkladu

Zadání instance: Máme 5 městských částí (O1–O5). Magistrát chce postavit 2 stanice rychlé pomoci tak, aby minimalizoval dojezdové časy k pacientům.

Vstupní data:

- Matice časů C :

$$C = \begin{bmatrix} 4 & 12 & 14 & 17 & 11 \\ 20 & 7 & 10 & 19 & 24 \\ 21 & 13 & 5 & 8 & 11 \\ 9 & 12 & 14 & 3 & 8 \\ 17 & 25 & 13 & 10 & 6 \end{bmatrix} \quad (55)$$

- Četnosti výjezdů f_j : O1: 30, O2: 50, O3: 42, O4: 36, O5: 24.

Ukázka a interpretace řešení: Předpokládejme, že výpočet v LINGO určil jako optimální stanoviště místa O2 a O4 ($y_2 = 1, y_4 = 1$).

- **Přiřazení (x_{ij}):** Stanice v O2 bude obsluhovat oblasti O1, O2 a O3. Stanice v O4 bude obsluhovat O4 a O5.
- **Výpočet účelové funkce:** $z = (c_{21} \cdot 30 + c_{22} \cdot 50 + c_{23} \cdot 42) + (c_{44} \cdot 36 + c_{45} \cdot 24)$
- **Slovní interpretace:** Vybudování stanic v těchto dvou bodech zajišťuje nejrychlejší možnou pomoc pro celé město vzhledem k tomu, jak často v jednotlivých částech lidé sanitku volají. Pokud by se stanice postavily jinde, průměrný čas čekání pacienta na pomoc by se zvýšil. Model také zajistil, že žádná stanice není přetížena (v příkladu omezeno na obsluhu max. 4 obvodů).

7 Úloha batohu (Knapsack Problem)

Úloha batohu je jednou z nejznámějších úloh celočíselného programování. Reprezentuje situaci, kdy se rozhodovatel snaží vybrat optimální kombinaci předmětů (činností, investic) do omezeného prostoru nebo rozpočtu tak, aby maximalizoval celkový přínos. V praxi se tento model využívá například při výběru investičních projektů, plnění nákladních prostor nebo optimalizaci výrobního portfolia.

7.1 Formulace matematického modelu

Základní model úlohy batohu předpokládá, že máme n předmětů, u kterých známe jejich váhu (spotřebu zdroje) a jejich užitek (hodnotu).

Rozhodovací proměnné:

- $x_i \in \{0, 1\}$: Binární proměnná, která nabývá hodnoty 1, pokud je předmět i vybrán do batohu, a 0 v opačném případě.

Parametry modelu:

- c_i : Užitek (cena, bodové hodnocení) i -tého předmětu.
- a_i : Váha (spotřeba kapacity) i -tého předmětu.
- b : Celková nosnost (kapacita) batohu.
- n : Počet dostupných předmětů.

Matematický zápis:

$$\max z = \sum_{i=1}^n c_i x_i \quad (56)$$

za podmínky:

$$\sum_{i=1}^n a_i x_i \leq b \quad (57)$$

$$x_i \in \{0, 1\}, \quad i = 1, 2, \dots, n \quad (58)$$

7.2 Interpretace matematického modelu

- **Účelová funkce:** Cílem je maximalizovat celkovou sumu užiteků vybraných předmětů. Sčítáme pouze ty hodnoty c_i , u kterých je příslušná proměnná x_i rovna 1.
- **Kapacitní omezení:** Součet vah všech vybraných předmětů nesmí překročit stanovenou nosnost b . Toto omezení zajišťuje, že vybrané řešení je fyzicky (nebo ekonomicky) realizovatelné.
- **Podmínka binární volby:** Omezuje rozhodování na "všechno, nebo nic". Předmět nelze do batohu vložit jen zčásti (např. v uříznuté podobě), což odlišuje úlohu batohu od úloh spojitého lineárního programování.

7.3 Formulace v jazyce LINGO

V programu LINGO se úloha batohu zapisuje pomocí sekcí **SETS** a **DATA**, což umožňuje snadnou změnu parametrů bez nutnosti přepisovat samotné rovnice.

MODEL:

SETS:

predmety /1..10/: x, c, a;

ENDSETS

DATA:

```

a = 4 5 3 2 1 5 3 2 4 3; ! Váhy predmetu;
c = 20 50 70 30 45 5 80 60 15 20; ! Užitky;
b = 15; ! Kapacita batohu;
ENDDATA

max = @SUM(predmety: c * x); ! Maximalizace celkového užitku;

@SUM(predmety: a * x) <= b; ! Kapacitní omezení;

@FOR(predmety: @BIN(x)); ! Definice binárních promenných;
END

```

Význam klíčových prvků:

- **@SUM:** Agregační funkce, která provádí sumaci přes definovanou množinu.
- **@BIN(x):** Příkaz, který vynucuje binární hodnoty pro proměnné x . Bez tohoto příkazu by LINGO řešilo úlohu jako spojitou, což by mohlo vést k nepřípustnému rozdělení předmětů.

7.4 Konkrétní číselný příklad a interpretace řešení

Zadání instance: Zákazník vybírá z 10 předmětů s cílem maximalizovat svůj užitek. Batoh má nosnost $b = 15$ kg.

Data příkladu:

- **Váhy a_i [kg]:** 4, 5, 3, 2, 1, 5, 3, 2, 4, 3
- **Užitky c_i [body]:** 20, 50, 70, 30, 45, 5, 80, 60, 15, 20

Formulace účelové funkce pro tento příklad:

$$\max z = 20x_1 + 50x_2 + 70x_3 + 30x_4 + 45x_5 + 5x_6 + 80x_7 + 60x_8 + 15x_9 + 20x_{10} \quad (59)$$

Formulace omezení pro tento příklad:

$$4x_1 + 5x_2 + 3x_3 + 2x_4 + 1x_5 + 5x_6 + 3x_7 + 2x_8 + 4x_9 + 3x_{10} \leq 15 \quad (60)$$

Interpretace možného řešení: Předpokládejme, že optimální řešení z LINGO vypíše $x_3 = 1, x_5 = 1, x_7 = 1, x_8 = 1, x_2 = 1$ a ostatní rovny nule.

- **Kontrola kapacity:** $3 + 1 + 3 + 2 + 5 = 14$ kg ≤ 15 kg (Podmínka splněna).
- **Výpočet užitku:** $z = 70 + 45 + 80 + 60 + 50 = 305$ bodů.

Slovní interpretace: Výsledek nám říká, že zákazník by měl nakoupit předměty č. 2, 3, 5, 7 a 8. Tato kombinace mu přinese nejvyšší možný subjektivní užitek (305 bodů), přičemž celková váha nákupu (14 kg) bezpečně vyhovuje nosnosti batohu. Pokud by se zákazník rozhodl přidat jakýkoliv další předmět (např. předmět č. 4 s váhou 2 kg), celková váha by překročila 15 kg a batoh by se rozbil. Model tedy našel nejlepší možnou rovnováhu mezi váhou a hodnotou.

8 Optimální cesty na síti mezi dvojicí uzlů

Hledání optimálních cest v grafu mezi startovním uzlem s a koncovým uzlem t je základní úlohou síťové analýzy. Optimalita se definuje podle zvoleného kritéria, kterým může být délka, čas, propustnost nebo jiná fyzikální charakteristika. Pro řešení těchto úloh se využívají algoritmy založené na principu extrémální algebry.

8.1 Základní algoritmy: Bellman-Ford a Dijkstra

1. Bellman-Fordův algoritmus (Základní algoritmus)

- **Princip:** Funguje na principu postupné relaxace všech hran. V každém kroku se snaží vylepšit odhad vzdálenosti do všech uzlů.
- **Použití:** Je univerzální. Lze ho použít pro nejkratší i nejdelší cesty a zvládá i záporné ohodnocení hran (pokud v grafu nevznikne záporný cyklus).

2. Dijkstrův algoritmus

- **Princip:** Jedná se o hladový algoritmus (greedy). V každém kroku vybere uzel s aktuálně nejmenším ohodnocením, označí jej za trvalý a přepočítá sousedy.
- **Rozdíly a omezení:** Je výrazně rychlejší než Bellman-Ford, ale **vyžaduje nezáporná ohodnocení hran**. V případě hledání nejdelší cesty nebo přítomnosti záporných hodnot selhává.

8.2 Kritéria optimality a extrémální algebra

Pro různé typy úloh se používá jednotný výpočetní rámec $d_j = \bigoplus (d_i \otimes c_{ij})$, kde $\bigoplus$ je operace pro výběr optimální hodnoty a $\otimes$ je operace pro skládání ohodnocení na cestě.

- **Nejkratší / Nejdelší cesta (Aditivní kritérium):**
 - **Princip:** Optimalita je dána součtem hodnot hran.
 - **Operace:** $\otimes$ je sčítání (+). Pro nejkratší cestu je $\bigoplus$ rovno MIN, pro nejdelší MAX.
 - **Význam:** Minimalizace nákladů, vzdálenosti nebo času (CPM).
- **Cesta s maximálním / minimálním zesílením (Multiplikativní kritérium):**
 - **Princip:** Ohodnocení cesty je dáno součinem hodnot hran (např. pravděpodobnost, zesílení signálu).
 - **Operace:** $\otimes$ je násobení ($\times$). $\bigoplus$ je MAX (nejsilnější signál) nebo MIN (minimální ztráta).
 - **Význam:** Přenos signálu, spolehlivost systému.
- **Cesta s maximální propustností (Kapacitní kritérium):**
 - **Princip:** Cesta je omezena svou „nejslabším článkem“ (bottleneck). Hledáme cestu, kde je toto minimum co největší.
 - **Operace:** $\otimes$ je MIN (vybíráme nejúžší hrdlo na cestě). $\bigoplus$ je MAX.
 - **Význam:** Kapacita potrubí, průtok dopravy.
- **Cesta s minimálním profilem (Kritérium kritické hodnoty):**
 - **Princip:** Ohodnocení cesty je dáno maximální hodnotou hrany na ní (např. nejvyšší stoupání). Chceme cestu, kde je toto maximum co nejmenší.
 - **Operace:** $\otimes$ je MAX (hledáme nejvyšší bod cesty). $\bigoplus$ je MIN.
 - **Význam:** Přeprava nadměrných nákladů pod mosty, minimalizace stoupání.

8.3 Souhrnná tabulka operací

Typ cesty	$\oplus$	$\otimes$	Neutrální prvek (d_1)	Neexistence hrany
Nejkratší	MIN	+	0	∞
Nejdelsí	MAX	+	0	$-\infty$
Max. zesílení	MAX	$\times$	1	0
Max. propustnost	MAX	MIN	∞ (resp. $\max c_{ij}$)	0
Min. profil	MIN	MAX	0	∞

8.4 Komplexní číselný příklad

Uvažujme orientovaný graf se 4 uzly (A, B, C, D) a následujícím ohodnocením hran c_{ij} :

- $A \rightarrow B$: 3, $A \rightarrow C$: 8
- $B \rightarrow C$: 4, $B \rightarrow D$: 5
- $C \rightarrow D$: 1

1. Hledání nejkratší cesty ($A \rightarrow D$):

- Cesta $A \rightarrow B \rightarrow D$: $3 + 5 = 8$
- Cesta $A \rightarrow B \rightarrow C \rightarrow D$: $3 + 4 + 1 = 8$
- Cesta $A \rightarrow C \rightarrow D$: $8 + 1 = 9$
- **Výsledek:** Optimální délka je 8 (trasy $A \rightarrow B \rightarrow D$ nebo $A \rightarrow B \rightarrow C \rightarrow D$).

2. Hledání cesty s maximální propustností ($A \rightarrow D$): Zde nás zajímá úzké hrdlo (minimum) na každé cestě:

- Cesta $A \rightarrow B \rightarrow D$: $\min(3, 5) = 3$
- Cesta $A \rightarrow B \rightarrow C \rightarrow D$: $\min(3, 4, 1) = 1$
- Cesta $A \rightarrow C \rightarrow D$: $\min(8, 1) = 1$
- **Výsledek:** Maximální propustnost je 3 (trasa $A \rightarrow B \rightarrow D$). Interpretace: I když je cesta delší, „protlačíme“ tudy více jednotek (např. aut).

3. Hledání cesty s minimálním profilem ($A \rightarrow D$): Zajímá nás nejvyšší hrana na cestě a tu chceme minimalizovat:

- Cesta $A \rightarrow B \rightarrow D$: $\max(3, 5) = 5$
- Cesta $A \rightarrow B \rightarrow C \rightarrow D$: $\max(3, 4, 1) = 4$
- Cesta $A \rightarrow C \rightarrow D$: $\max(8, 1) = 8$
- **Výsledek:** Minimální profil je 4 (trasa $A \rightarrow B \rightarrow C \rightarrow D$). Interpretace: Tato cesta je nejvhodnější pro náklad, který nesmí překročit určitou výšku/váhu.

4. Hledání cesty s maximálním zesílením ($A \rightarrow D$): Předpokládejme, že hodnoty hran jsou koeficienty (např. $A \rightarrow B = 0.3$ atd.):

- Cesta $A \rightarrow B \rightarrow D$: $0.3 \times 0.5 = 0.15$
- Cesta $A \rightarrow C \rightarrow D$: $0.8 \times 0.1 = 0.08$
- **Výsledek:** Trasa přes uzel B je výhodnější, výsledný signál bude silnější.